

</> PYTHON + OBJECT-ORIENTED PROGRAMMING

Employee Performance Prediction System

A pure Python rule-based evaluation system built using Object-Oriented Programming principles to analyze employee metrics and predict performance levels.

 **Lakshmi Tirupathamma**
Project Presenter & Developer

Project Overview & Objectives

🎯 Project Overview

- **Core Purpose:** Evaluates employee performance using CSV datasets and structured Python OOP design.
- **No ML Libraries:** Built without Pandas, NumPy, or Scikit-learn to emphasize core OOP architecture.
- **Weighted Scoring:** Calculates individual performance ratings based on key workforce productivity indicators.
- **Classification:** Categorizes staff into 4 distinct rating levels: *Excellent, Good, Average, Poor*.

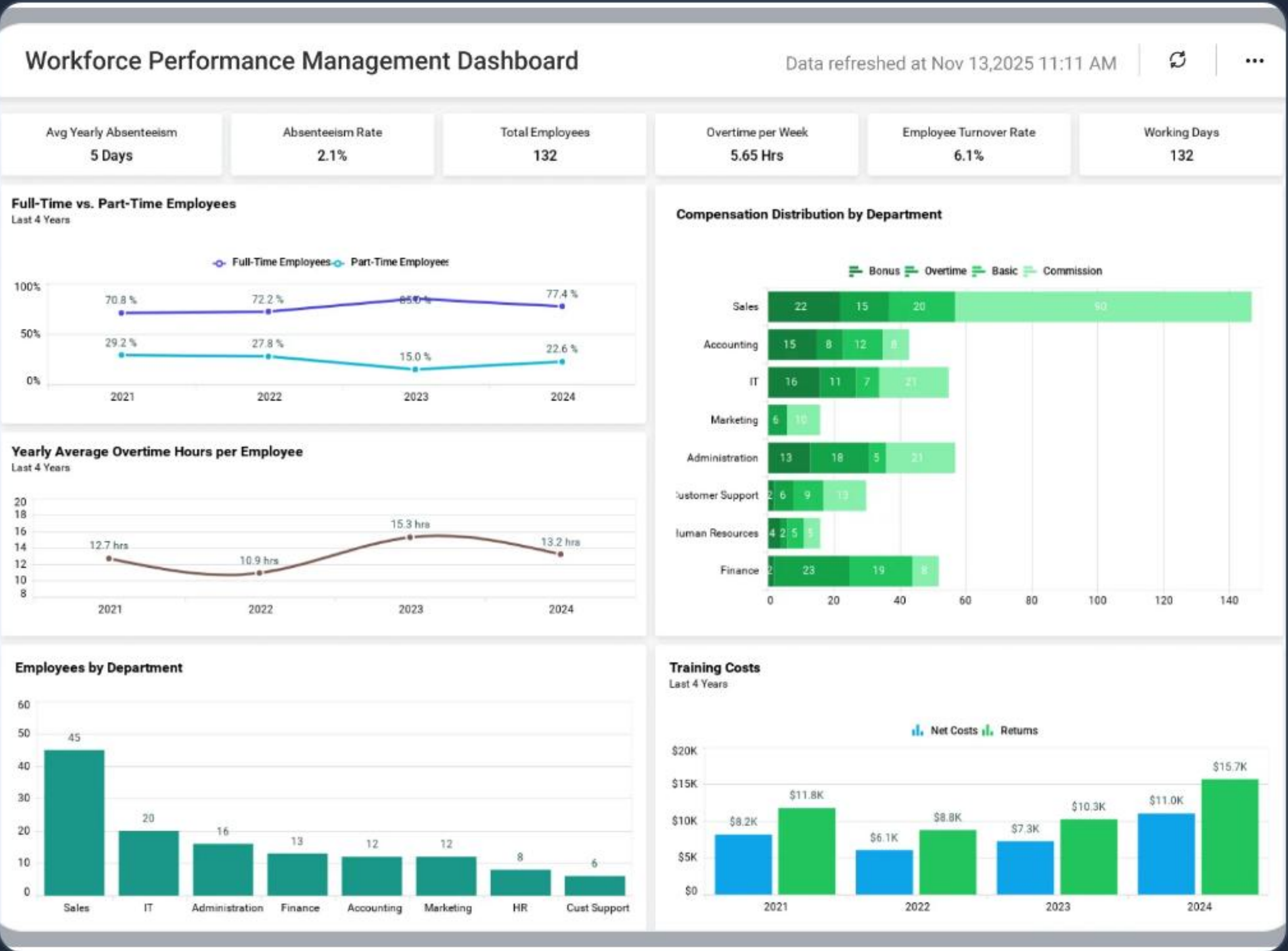
☰ Key Objectives

- Store and manipulate employee information using modular classes.
- Read and parse CSV dataset records seamlessly with Python's built-in csv module.
- Demonstrate practical implementation of all 4 major OOP pillars:
Abstraction **Encapsulation** **Inheritance** **Polymorphism**
- Provide a clean, maintainable, and understandable software design.

Dataset & Performance Factors

Weighted Factors (Total: 100%)

Factor Name	Weightage	Description
Attendance	20%	Presence percentage (0 - 100%)
Projects Completed	20%	Delivered project count
Previous Score	25%	Historical performance baseline
Job Satisfaction	15%	Employee survey rating (1 - 5)
Training Hours	10%	Skill development hours
Work Hours	10%	Average daily / weekly hours



OOP Concept 1: Abstraction

Concept & Implementation

Abstraction hides complex underlying implementation details and exposes only essential interfaces to the user.

- Utilizes Python's built-in abc module (ABC, @abstractmethod).
- Defines the base class Performance as a contract.
- Forces child classes to implement predict_performance().
- **Benefit:** Standardizes behavior across evaluation algorithms while preventing direct instantiation of incomplete base classes.

```
from abc import ABC, abstractmethod

# Abstract Base Class
class Performance(ABC):
    @abstractmethod
    def predict_performance(self):
        pass # Enforces interface

# Concrete Implementation
class EmployeePerformance(Performance):
    def predict_performance(self):
        return "Excellent"

# Usage
obj = EmployeePerformance()
print(obj.predict_performance())
# Output: Excellent
```

OOP Concept 2: Encapsulation

Concept & Implementation

Encapsulation wraps data (attributes) and methods inside a single class while restricting direct external access.

- Uses double underscores (e.g., `self.__attendance`) to make attributes private.
- Provides public **Getter** methods to safely read data.
- Provides public **Setter** methods with validation logic to prevent invalid inputs (e.g., attendance out of 0-100% range).
- **Benefit:** Protects internal state integrity.

```
class Employee:
    def __init__(self, emp_id, name, attendance):
        self.__emp_id = emp_id          # Private
        self.__name = name              # Private
        self.__attendance = attendance

    def get_name(self):
        return self.__name

    def get_attendance(self):
        return self.__attendance

    def set_attendance(self, value):
        if 0 <= value <= 100:
            self.__attendance = value

emp = Employee(101, "Ravi", 95)
print(emp.get_name(), emp.get_attendance())
```

OOP Concept 3: Inheritance

Concept & Implementation

Inheritance allows a child class to acquire attributes and methods from a parent class, eliminating code duplication.

- **Parent Class:** Employee stores general profile attributes (ID, Name, Attendance, etc.).
- **Child Class:** PerformanceCalculator inherits Employee.
- Extends functionality by adding specialized scoring and calculation logic.
- **Benefit:** High code reusability and clean modular hierarchy.

```
# Parent Class
class Employee:
    def __init__(self, name, attendance):
        self.name = name
        self.attendance = attendance

# Child Class inheriting Employee
class PerformanceCalculator(Employee):
    def calculate_score(self):
        # Reuses self.attendance from Parent
        return self.attendance * 0.20

emp = PerformanceCalculator("Ravi", 95)
print(emp.name, emp.calculate_score())
# Output: Ravi 19.0
```

OOP Concept 4: Polymorphism

📁 Concept & Implementation

Polymorphism enables different classes to implement methods with the exact same name, allowing unified interface calls.

- Demonstrated through **Method Overriding**.
- Subclasses (HighPerformance, LowPerformance) override `predict_performance()`.
- Iterating over a list of objects allows invoking the same method call to trigger different class-specific logic.
- **Benefit:** Flexibility and dynamic runtime evaluation.

```
class PerformanceCalculator:
    def predict_performance(self):
        return "Good"

class HighPerformance(PerformanceCalculator):
    def predict_performance(self):
        return "Excellent"

class LowPerformance(PerformanceCalculator):
    def predict_performance(self):
        return "Needs Improvement"

employees = [HighPerformance(), LowPerformance()]
for emp in employees:
    print(emp.predict_performance())  # Polymorphic
```

Scoring Logic & Classification

```
def calculate_score(self):
    attendance = self.attendance * 0.20
    projects = min(self.projects * 10, 100) * 0.20
    training = min(self.training_hours * 2, 100) * 0.10
    work = min(self.work_hours * 10, 100) * 0.10
    satisfaction = self.satisfaction * 20 * 0.15
    previous = self.previous_score * 0.25

    total = (attendance + projects + training +
            work + satisfaction + previous)
    return round(total, 2)
```

Classification Thresholds

Score Range	Performance Rating
80 – 100	Excellent
65 – 79	Good
50 – 64	Average
Below 50	Poor

Evaluation & Future Scope



Advantages

- Built purely on standard Python without heavy external dependencies.
- Clean demonstration of all core Object-Oriented principles.
- Modular and easily extensible scoring rules.



Limitations

- Rule-based deterministic scoring rather than Machine Learning predictive models.
- Relies on static CSV files without live database storage.
- Command-line output only.



Future Scope

- Integration with Tkinter GUI / Web Interface.
- Database connection (SQLite/MySQL).
- Incorporating Scikit-Learn ML models for dynamic prediction.



Thank You!

Employee Performance Prediction System (Python + OOP)

Presented by: **Lakshmi Tirupathamma**



Questions & Discussion

Image Sources



<https://www.boldbi.com/wp-content/uploads/2025/12/workforce-performance-management-dashboard-v2-1.webp>

Source: www.boldbi.com